

Java – Beautifully Broken

Equal but lost: the HashMap that forgot your key

Issue #3

Sat, 03 Oct 2026

⌚ ~18 min read

Level 3/5

⚡ The short version

1. `equals`, `hashCode` and `compareTo` are contracts, and `HashMap`, `HashSet` and `TreeSet` trust you to keep them. Break one and nothing throws: your key just quietly disappears.
2. Free Oracle JDK 21 updates end this month. From the 20 October Critical Patch Update, Oracle JDK 21 updates move to the OTN licence. Oracle's advice: move to JDK 25.
3. Quarkus 4.0.0.Beta1 is out, with a Java 21 baseline, Hibernate ORM 8, Jackson 3 and opt-in HTTP/3. GA is planned for the end of November.

Version watch

25

current LTS (Sep 2025),
free Oracle updates until
Sep 2028

27

latest JDK, GA September
2026 (non-LTS)

4.1.1

latest Spring Boot GA (20
Aug 2026); 4.2.0-M2 in
milestones

29

next planned LTS,
September 2027

Sources: Oracle Java SE support roadmap (updated 15 Sep 2026), Spring Boot releases (checked 3 Oct).

PART 1 Learn



`equals`, `hashCode`, `compareTo`, and how HashMap really works

REVISE

~10 MIN

LEVEL 3

Every cache, dedup step and `groupBy` you write leans on three methods you probably last thought hard about in an interview. When they're wrong, nothing crashes. A `get` returns `null`, a `Set` holds a "duplicate", a `TreeSet` drops an item. These bugs survive code review because each method looks fine on its own. The failure only appears when a collection uses them together.

1 · THE TWO CONTRACTS, IN ONE BREATH

The `Object` Javadoc spells out both. `equals` must be an equivalence relation: reflexive, symmetric, transitive, consistent over time, and `x.equals(null)` is `false`. `hashCode` has one hard rule: equal objects must have equal hash codes. Unequal objects may share a hash code. That's legal, just slow. (Oracle, `Object` Javadoc)

The Javadoc adds the line that causes most real bugs: if you override `equals`, you generally must override `hashCode` too. (Oracle) A hash table checks the hash first and only calls `equals` on keys that land in the same bucket. Two "equal" objects with different identity hashes almost never meet.

A map key: naive vs correct

Java 8: equals only, mutable

```
class OrderKey {
    String customerId; // not final
    LocalDate day;
    @Override
    public boolean equals(Object o) {
        if (!(o instanceof OrderKey)) return false;
        OrderKey k = (OrderKey) o;
        return customerId.equals(k.customerId)
            && day.equals(k.day);
    }
    // no hashCode(): identity hash
}
```

Java 16+: a record

```
record OrderKey(String customerId,
                LocalDate day) {
    OrderKey {
        Objects.requireNonNull(customerId);
        Objects.requireNonNull(day);
    }
    // equals, hashCode, toString are
    // generated from all components,
    // and every field is final.
}
```

We compiled both on JDK 21. On the left, `set.contains(new OrderKey(...))` returns `false` for a key you just added (barring a lucky hash match), because each instance hashes by identity. On the right, the same lookup works. Records get the full topic in two issues' time.

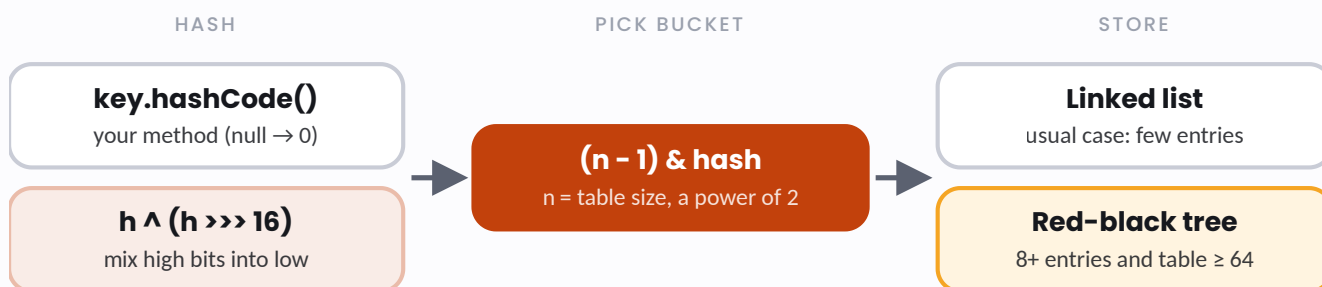
When you can't use a record (a JPA entity, a class with a superclass, a legacy DTO), write the pair by hand and keep two rules:

- **Use the same fields in both methods.** If `equals` compares `customerId` and `day`, `hashCode` hashes exactly those, for example `Objects.hash(customerId, day)`. A field in `hashCode` but not in `equals` breaks the contract outright. A field in `equals` but not in `hashCode` is legal, but costs you collisions.
- **Choose `getClass()` or `instanceof` on purpose.** `instanceof` lets a subclass equal its parent, and the moment the subclass adds a field to its own `equals`, symmetry breaks: `parent.equals(child)` is `true`, `child.equals(parent)` is `false`. Make the class `final`, or compare `getClass()`. Note that Hibernate proxies are subclasses, which is why ORM-friendly `equals` methods usually use `instanceof`.

2 · WHAT `PUT` ACTUALLY DOES

A `HashMap` is an array of buckets, and your `hashCode` decides which bucket. Here's the path of one `put(key, value)`, confirmed with `javap` on JDK 21.

One `put(key, value)`, step by step



Constants from `javap -constants java.util.HashMap`: default capacity 16, load factor 0.75, `TREEIFY_THRESHOLD` 8, `UNTREEIFY_THRESHOLD` 6, `MIN_TREEIFY_CAPACITY` 64. These are implementation details, not API, and can change.

Three things fall out of that picture:

- **The XOR step exists because the bucket index uses only the low bits.** With 16 buckets, only the bottom 4 bits of the hash matter. Folding the high 16 bits down means a `hashCode` that varies only in its high bits still spreads out.
- **Resizing doubles the table.** When the entry count passes $\text{capacity} \times 0.75$, the map rebuilds with about twice the buckets. (Oracle, `HashMap` Javadoc) Because the size is a power of two, each entry either stays at its index or moves up by exactly the old capacity, which makes the split cheap.
- **Pre-size when you know the count.** `new HashMap<>(1000)` is a classic trap: 1000 is a *capacity*, rounded up to 1024, so the map still resizes after 768 entries. Since Java 19, `HashMap.newHashMap(1000)` picks a capacity that fits 1000 mappings without resizing. (Oracle)

3 · TREEIFICATION: JAVA 8'S INSURANCE POLICY

Before Java 8, a bucket was always a linked list, so many keys with the same hash made lookups $O(n)$. JEP 180 changed that: once a bucket grows past a threshold, it switches to a balanced tree, improving the worst case to $O(\log n)$. (JEP 180) The tree is ordered by hash and, when keys are `Comparable`, by `compareTo` to break ties. That's why the Javadoc says comparable keys help when "many keys" share a hash. (Oracle)

The `MIN_TREEIFY_CAPACITY` of 64 is a nice detail. In a small table, a long bucket more likely means "too few buckets" than "bad hash", so **the map resizes first and only builds trees once the table has at least 64 buckets**. A tree that shrinks to 6 entries turns back into a list.

In the real world: the 2011 hash-flooding attacks

In December 2011, Alexander Klink and Julian Wälde showed that attackers could send **HTTP POST requests full of parameter names chosen to share one hash code**. Servers parse parameters into a hash table, so each request turned into a quadratic pile of collisions. oCERT's advisory listed Java, **Tomcat up to 7.0.22, Jetty up to 7.5.4 and GlassFish up to 3.1.1** among those affected, and warned that one request could keep a CPU at 100% for a long time with very little attacker bandwidth. (oCERT-2011-003) Java's `String.hashCode` is easy to collide on purpose: "Aa" and "BB" both hash to 2112, and you can combine such pairs into thousands of colliding strings. JEP 180's tree bins, shipped in Java 8, cap the damage at $O(\log n)$ per lookup, because `String` is `Comparable`. (JEP 180)

4 · COMPARETO : THE THIRD CONTRACT

`TreeMap` and `TreeSet` never call `equals`. They decide "same key" with `compareTo` (or your `Comparator`). The `Comparable` contract asks for sign symmetry and transitivity, and strongly recommends being **consistent with equals**: `compareTo` returns 0 exactly when `equals` returns `true`. Break that and sorted collections behave, in the Javadoc's word, "strangely". (Oracle, `Comparable` Javadoc)

The JDK's own famous exception is `BigDecimal`: `4.0` and `4.00` compare as equal but aren't `equals`. (Oracle) We checked on JDK 21: put both in a `HashSet` and you get **2 elements**; put both in a `TreeSet` and you get **1**.

Sorting items by price: naive vs correct

Naive: subtraction, no tie-break

```
Comparator<Item> byPrice =  
    (a, b) -> a.cents() - b.cents();  
  
// overflows: MIN_VALUE - 1 wraps  
// to MAX_VALUE, so "less" becomes  
// "greater" and sorting breaks  
  
var set = new TreeSet<Item>(byPrice);  
set.add(new Item("pen", 100));  
set.add(new Item("cap", 100));  
set.size(); // 1: "cap" was dropped
```

Correct: comparing + thenComparing

```
Comparator<Item> byPrice =  
    Comparator.comparingInt(Item::cents)  
        .thenComparing(Item::name);  
  
var set = new TreeSet<Item>(byPrice);  
set.add(new Item("pen", 100));  
set.add(new Item("cap", 100));  
set.size(); // 2: ties broken by name
```

`record Item(String name, int cents)`. Both panes compiled and ran on JDK 21: the left comparator returns 2147483647 for (MIN_VALUE, 1). The right one is consistent with the record's equals, because it compares every component.

💡 Gotcha: the key you mutated after put

The hash is computed **once, at put time**, and stored with the entry. Change a field that `hashCode` uses afterwards, and the entry sits in the bucket for its *old* hash. We tried it on JDK 21: `map.get(key)` with the very same object returns `null`, `containsKey` with a fresh copy of the old value returns `false`, and `size()` still says 1. The entry is unreachable until it's removed by iteration. The `Object` contract only promises a stable hash "provided no information used in `equals` comparisons on the object is modified". (Oracle) The fix is boring and total: **make keys immutable**. A record with immutable components is the easiest way. Watch out for JPA entities with a generated `@Id` in `hashCode`: the id changes from `null` to a value on persist.

🔍 Under the hood

- **String caches its hash** in a field after the first call, so repeated lookups with the same string don't rehash it. That's one reason strings make good keys.
- **Record equals and hashCode are generated through invokedynamic** (`ObjectMethods.bootstrap`, as `javap -v` shows), not written into the class file as code, so the JDK can improve them without recompiling your classes. Note a record's `equals` uses each component's own `equals`, so **a record holding a `BigDecimal` inherits the 4.0 vs 4.00 problem**. We checked: `new Money(new BigDecimal("4.0")).equals(new Money(new BigDecimal("4.00")))` is `false`.
- **HashMap isn't thread-safe**. Unsynchronized concurrent writes can corrupt it, and its iterators are fail-fast, throwing `ConcurrentModificationException` on a best-effort basis. (Oracle) Use `ConcurrentHashMap` for shared maps.

Try it (15 minutes, jshell on JDK 17+)

1. **Lose a key.** Define `class K { int id; K(int i){id=i;} public boolean equals(Object o){return o instanceof K k && k.id==id;} public int hashCode(){return id;} }`. Put `k = new K(1)` into a `HashMap`, set `k.id = 2`, then try `get(k)` and `get(new K(1))`. Print `size()`.
2. **BigDecimal split personality.** Add `new BigDecimal("4.0")` and `new BigDecimal("4.00")` to a `HashSet` and a `TreeSet`. Compare sizes. Then try `stripTrailingZeros()` on both before adding.
3. **Watch treeification pay off.** Make record `Bad(int id)` with `hashCode()` always returning 42. Put 20,000 instances in a `HashMap` and time 20,000 `get` s. Then add `implements Comparable<Bad>` (compare by `id`) and time again. In one rough run on our JDK 21 sandbox with 50,000 keys, the gets took **about 17 seconds without Comparable** and **about 40 ms with it**. Not a benchmark, but the gap is JEP 180 at work.

WHY IT MATTERS TO YOU



Day-to-day impact



Interview value



Act-now urgency

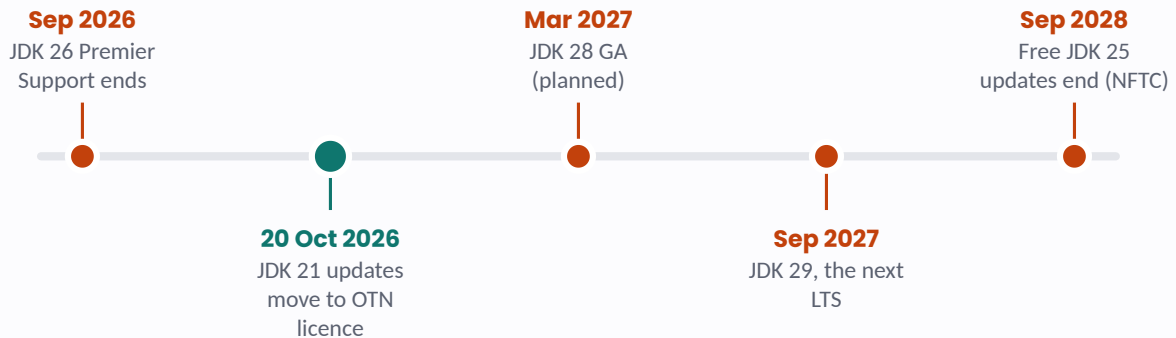


GO DEEPER

- `Object` and `Comparable` (Oracle Javadoc). The contracts, word for word.
- `HashMap` (Oracle Javadoc). Load factor, iteration cost and `newHashMap`.
- **JEP 180: Handle Frequent HashMap Collisions with Balanced Trees** (OpenJDK). Why buckets became trees.
- **oCERT-2011-003**. The hash-flooding advisory, with affected versions.

~8 min • A quiet couple of days for releases, so we go deeper on two things you'll feel this quarter: an Oracle licence deadline, and Quarkus's next major version.

Oracle JDK: the dates that matter now



Sources: Oracle Java SE support roadmap, Oracle security alerts schedule, Oracle Java blog.

Top story 1 • Free Oracle JDK 21 updates end with the 20 October patch

ACT THIS MONTH

LICENSING

What happened. Oracle has been shipping JDK 21 under its **No-Fee Terms and Conditions (NFTC)** licence, which allows free production use. That free period has ended. In Oracle's words, further Oracle JDK 21 updates, "**beginning with the October 2026 Critical Patch Update**", are planned under the **Java SE OTN licence**, the same model that already covers Oracle's Java 8, 11 and 17 updates. (Oracle Java blog) The October CPU is scheduled for **Tuesday 20 October 2026**. (Oracle security alerts)

The support roadmap now states the same thing, and adds that **JDK 25 stays under the free licence through September 2028**. Java 21 itself is far from dead: paid Premier Support runs to September 2028 and Extended Support to September 2031. (Oracle roadmap)

● Oracle JDK 21 after 20 Oct

- Updates under the Java SE OTN licence
- Same model as Oracle's Java 8, 11 and 17 updates
- Old NFTC builds keep working, but get no new security fixes

VS

● Oracle JDK 25

- Current LTS, free under NFTC until Sep 2028
- Oracle's recommended move
- Premier Support to Sep 2030

Why it matters. This is the "free LTS for three years" model doing exactly what it said, but it catches teams that adopted Oracle's JDK 21 images or installers in 2023 and never revisited the choice. From the October CPU on, **staying patched on Oracle's JDK 21 binaries means accepting the OTN terms**, so read them before your next patch run. Oracle recommends moving to JDK 25 and points to its Java SE Universal Subscription for

anyone who needs to stay. (Oracle Java blog) The change is about **Oracle's binaries**. OpenJDK builds from other distributors have their own licences and support schedules, so check which one you actually run.

The usual hiding places are not the servers you think about. They're the **Dockerfile** **FROM** line copied from an **old template**, the CI image that builds your jars, the desktop tool someone installed with the Oracle installer, and the vendored JRE inside an on-prem product. A licence change is also a good moment to ask the bigger question: **JDK 25 has been out for a year**, Spring Boot 4 and Quarkus 4 both run on it, and the jump only gets bigger the longer you wait.

 **What to do.** Run `java -version` on your servers and check your base images: the vendor string tells you whose build it is. If it's **Oracle JDK 21 in production**, pick one path before 20 October: move to **JDK 25**, switch to another vendor's JDK 21 build, or budget for a subscription. Don't just stop patching: the October CPU will carry security fixes.

WHY IT MATTERS TO YOU



Day-to-day impact



Act-now urgency



Interview value



Top story 2 · Quarkus 4.0.0.Beta1: Java 21 baseline and a big dependency jump

FRAMEWORKS

TRY THE BETA

What happened. On 1 October, the Quarkus team released **Quarkus 4.0.0.Beta1**, the first beta of its next major version. **Java 21 is now the minimum**, and the team says virtual threads become more central to the framework. GA is planned for **the end of November**. (Quarkus blog)

Almost every major dependency moves up a generation (Quarkus blog):

Area	Quarkus 4
Persistence	Hibernate ORM 8 , Jakarta Persistence 4.0, Jakarta Data 1.1
JSON	Jackson 3
Networking	Vert.x 5, Netty 4.2
APIs	Jakarta REST 4, CDI 5.0

New features include **QUIC support** as the base for HTTP/3, MQTT over QUIC and WebTransport, **opt-in HTTP/3** with auto-generated certificates in dev and test, native epoll or io_uring transports, an **Observation API** for metrics and tracing (`@Observed`), a JMS extension compatible with Spring JMS, and an `@ExactlyOnce` annotation for Kafka transactions. (Quarkus blog)

Why it matters. Two of those upgrades will touch your code, not just your `pom.xml`. **Jackson 3** moves from `com.fasterxml.jackson` to `tools.jackson` packages (annotations excepted) and flips defaults: properties sort alphabetically and dates serialize as ISO-8601 strings instead of timestamps. Spring Boot 4 already made that same jump. (spring.io) **Hibernate ORM 8** brings Jakarta Persistence 4.0. The **Java 21 floor** fits a wider trend: frameworks now assume virtual threads exist.

 **What to do.** On Quarkus 3.x: create a branch, bump to the beta, and **run your test suite** to find the Jackson 3 and Hibernate 8 breakages early. If you maintain a Quarkus extension, the team is explicitly asking extension authors to migrate now and to report problems as GitHub issues. Everyone else can wait for GA.

WHY IT MATTERS TO YOU



Day-to-day impact



Act-now urgency



Interview value



Quick hits

Quarkus Desktop.

A new Quarkiverse extension (0.1.0) builds **AWT and Swing apps** with CDI injection and config, in JVM mode or as GraalVM native executables on Windows, Linux and macOS. macOS native builds need Quarkus 4.0 and GraalVM 25.1+. ([Quarkus blog](#))

JobRunr 9.

The background-job library adds timeline and Gantt views of job history, Micronaut 5 support and Quarkus 3.40 LTS compatibility. It's the **last major version whose open-source edition supports Java 8**. Breaking changes include `SmartQueue` renamed to `JobPrefetchQueue`. ([JobRunr migration guide](#)) ([JVM Weekly](#))

Spring Boot 4.2 RC1 notes are taking shape.

The draft release notes say the default image builder for `bootBuildImage / spring-boot:build-image` becomes `paketobuildpacks/ubuntu-resolute-builder`. If you pin builders in CI, take note. RC1 itself isn't out yet. ([Spring Boot wiki](#))

JDK 26 is out of Oracle support.

As a non-LTS release, its Premier Support ended in September 2026 when JDK 27 shipped. On 26? Move to 27, or back to 25 LTS. ([Oracle roadmap](#))

Jakarta EE 12 inches forward.

CDI 5.0 passed its ballot on 11 September. The Core Profile is planned before a 1 December livestream, and the Web Profile for Q1 2027. CDI 5.0 is what Quarkus 4 already builds on. ([JVM Weekly](#))

Leyden keeps chipping at startup.

JVM Weekly reports an iterative AOT-training prototype that took one app's startup from about 0.25 s to under 0.1 s. It's a prototype, not a JEP, so treat it as direction, not a promise. ([JVM Weekly](#))

Maven 4.0 still at RC7.

No GA yet. The team called RC7 "expected to be the last before GA". ([JVM Weekly](#))

Coming up.

Oracle's Critical Patch Update on **20 October**, Quarkus 4.0 GA at the end of November, Jakarta EE 12 Core Profile before 1 December, and JDK 28 in March 2027.

Revision card

- Override `equals`, override `hashCode`. Equal objects must have equal hashes; unequal ones may collide.
- Keys must be immutable. The hash is stored at `put` time; mutate a key and the entry becomes unreachable.
- `HashMap` = power-of-two bucket array. Index is `(n-1) & (h ^ h>>16)`; it doubles when size passes capacity $\times 0.75$.
- Buckets become trees at 8 entries once the table has 64+ buckets (JEP 180, Java 8). `Comparable` keys help.
- Use `HashMap.newHashMap(n)` (Java 19+) to pre-size; `new HashMap<>(n)` sets capacity, not expected size.
- `TreeSet` / `TreeMap` use `compareTo` only. Keep it consistent with `equals`; never compare by subtraction.
- Oracle JDK 21: updates from the 20 Oct CPU move to the OTN licence. JDK 25 is free until Sep 2028.
- Quarkus 4 beta: Java 21 baseline, Hibernate ORM 8, Jackson 3. GA planned for end of November.

Java – Beautifully Broken is researched and written by Claude every morning from OpenJDK, Oracle and Spring announcements, official docs, InfoQ and engineering blogs. Vendor numbers are labelled as claims, and every item links to its source. Want a topic sooner or a different depth? Change `java/settings.yml` or just reply. ☕